

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

Khoa Công nghệ Thông tin



BÁO CÁO ĐỒ ÁN
PHÂN TÍCH THÀNH PHẦN CHÍNH (PCA)

Học phần **CSC16003 – PHÂN TÍCH THỐNG KÊ DỮ LIỆU NHIỀU BIẾN**

Lớp **22TGMT**

Giảng viên **Lý Quốc Ngọc**

Nguyễn Mạnh Hùng

Phạm Thanh Tùng

Sinh viên **22127318 – Trang Minh Nhật**

22127167 – Trương Học Huy

22127402 – Bé Lã Anh Thư

TP. Hồ Chí Minh, 2025

Mục lục

1	Giới thiệu	3
1.1	Bối cảnh thực hiện	3
1.2	Ý nghĩa khoa học	3
1.3	Ý nghĩa thực tiễn	4
1.4	Phát biểu bài toán	4
1.5	Phương pháp phân tích thành phần chính	5
1.5.1	Sử dụng đại số tuyến tính	6
1.5.2	Tiếp cận tối ưu hóa Lagrange	8
1.5.3	Trực quan hóa	10
1.6	Giải thuật	13
1.7	Đóng góp	14
1.8	Thách thức	15
2	Các công trình nghiên cứu liên quan	16
2.1	Tổng quan các biến thể PCA	16
2.2	So sánh các phương pháp	17
3	Phương pháp	18
3.1	Mục tiêu	19
3.2	Phương pháp thực hiện	19
3.2.1	Ánh xạ dữ liệu vào không gian đặc trưng	19
3.2.2	Tính ma trận kernel	20
3.2.3	Tâm hóa dữ liệu trong không gian đặc trưng	20
3.2.4	Giải bài toán trị riêng	21
3.2.5	Chuẩn hóa các vector riêng	21
3.2.6	Chiếu dữ liệu vào không gian mới	21
4	Cài đặt và kết quả thực nghiệm	22
4.1	Mô tả Bộ dữ liệu (Datasets)	22
4.1.1	Olivetti Faces	22
4.1.2	LFW (Labeled Faces in the Wild) - Subset	22

4.2	Giải thuật và Code Tương ứng	23
4.2.1	Tiền xử lý	23
4.2.2	PCA Tuyến tính (Cài đặt bằng SVD)	24
4.2.3	Kernel PCA (KPCA)	24
4.3	Giải thích Độ đo (k-NN CV Accuracy)	27
4.3.1	Bộ phân loại k-NN (k-Nearest Neighbors - k-Láng giềng Gần nhất)	27
4.3.2	Kỹ thuật Cross-Validation (CV - Đánh giá chéo)	27
4.3.3	Độ chính xác k-NN CV Trung bình	28
4.4	Trình bày Kết quả và Nhận xét	29
5	Kết luận và hướng phát triển	31
5.1	Kết luận	31
5.2	Hướng phát triển	31

1 Giới thiệu

1.1 Bối cảnh thực hiện

Trong kỷ nguyên dữ liệu lớn và sự phát triển mạnh mẽ của trí tuệ nhân tạo, việc xử lý và phân tích dữ liệu đa chiều đóng vai trò ngày càng quan trọng. Trong các lĩnh vực như thị giác máy tính, tài chính, sinh học và học máy, khối lượng dữ liệu thu thập được không ngừng gia tăng, đòi hỏi những phương pháp hiệu quả để xử lý và trích xuất thông tin phục vụ cho các tác vụ cụ thể.

Mặc dù dữ liệu lớn mang lại nhiều lợi ích, nhưng không phải lúc nào toàn bộ dữ liệu cũng phù hợp với mọi tác vụ. Do đó, quá trình xử lý và làm sạch dữ liệu trở nên cần thiết. Một trong những kỹ thuật được áp dụng phổ biến là giảm chiều dữ liệu, bởi dữ liệu có số chiều cao thường gây ra nhiều thách thức như tăng độ phức tạp tính toán, giảm hiệu suất mô hình và khó khăn trong việc trực quan hóa.

Phân tích thành phần chính (PCA - Principal Component Analysis) là một trong những phương pháp phổ biến nhất để giảm chiều dữ liệu mà vẫn giữ lại được thông tin quan trọng. **PCA hoạt động bằng cách biến đổi một tập dữ liệu có số chiều cao thành một tập mới với số chiều thấp hơn, đảm bảo bảo toàn tối đa phương sai của dữ liệu.** Phương pháp này xác định các thành phần chính (principal components), tức là các hướng trong không gian dữ liệu có phương sai lớn nhất, và chiếu dữ liệu lên các hướng đó nhằm giảm thiểu **mất mát thông tin**. Nhờ đó, PCA giúp cải thiện hiệu suất của các mô hình học máy, giảm nhiễu và tối ưu hóa việc trực quan hóa dữ liệu.

1.2 Ý nghĩa khoa học

PCA là một phương pháp thống kê được sử dụng rộng rãi để giảm chiều dữ liệu trong các bài toán phân tích dữ liệu lớn.

Về mặt lý thuyết, PCA *tìm một không gian con mới có số chiều thấp hơn* ($n < p$, với p là số chiều ban đầu của dữ liệu) sao cho khi chiếu dữ liệu xuống không gian này, các thành phần chính thu được *không còn tương quan tuyến tính* với nhau. Điều này đồng nghĩa với việc dữ liệu sau khi chuyển đổi sẽ có *phương sai được bảo toàn nhiều nhất có thể*, giúp cải thiện tính khả tích của dữ liệu.

Tính chất này giúp PCA trở thành một công cụ quan trọng trong nhiều lĩnh vực khoa học, đặc biệt trong **xử lý tín hiệu, thị giác máy tính, sinh học tính toán và phân tích dữ liệu tài chính**.

Bằng cách giảm số chiều nhưng vẫn giữ lại các thông tin quan trọng, PCA hỗ trợ việc **phân tích, trực quan hóa** và xử lý dữ liệu hiệu quả hơn, đồng thời giúp **giảm hiện tượng quá khớp (overfitting)** trong các mô hình học máy.

1.3 Ý nghĩa thực tiễn

Trong thực tế, PCA có nhiều ứng dụng quan trọng trong các lĩnh vực khác nhau.

- **Nhận diện khuôn mặt**, nơi mỗi ảnh có thể chứa hàng triệu pixel, dẫn đến số chiều dữ liệu rất lớn. PCA giúp giảm số chiều xuống vài trăm (thay vì vài triệu như ban đầu) trong khi vẫn giữ lại được những đặc trưng quan trọng nhất của khuôn mặt, từ đó cải thiện hiệu suất tính toán và giảm chi phí lưu trữ. Ứng dụng này được triển khai trong phương pháp Eigenfaces, một thuật toán nhận diện khuôn mặt nổi tiếng sử dụng PCA để trích xuất các thành phần chính của khuôn mặt và xây dựng không gian đặc trưng tối ưu cho việc phân loại.
- Ngoài ra, trong lĩnh vực **tài chính**, PCA được sử dụng để phân tích dữ liệu thị trường chứng khoán, giúp giảm số lượng biến số đầu vào trong các mô hình dự đoán, qua đó loại bỏ những biến dư thừa và tối ưu hóa quá trình ra quyết định đầu tư. Trong **y sinh học**, PCA cũng được ứng dụng để phân tích dữ liệu gene, giúp xác định các đặc trưng quan trọng trong chẩn đoán ung thư và các bệnh di truyền.

Nhờ khả năng tối ưu hóa xử lý dữ liệu đa chiều, PCA là một công cụ không thể thiếu trong các bài toán phân tích dữ liệu thực tiễn, đặc biệt là khi làm việc với dữ liệu lớn và có cấu trúc phức tạp.

1.4 Phát biểu bài toán

Nêu vấn đề Mục tiêu của Phân tích Thành phần Chính (PCA) là tìm cho ra một tập hợp các biến mới, được gọi là các thành phần chính (principal components) sao cho:

- Các thành phần chính là **các tổ hợp tuyến tính** của các biến gốc, và chúng **không tương quan tuyến tính** với nhau.

- Thành phần chính đầu tiên là tổ hợp tuyến tính của các biến gốc sao cho **phương sai của dữ liệu theo hướng đó là lớn nhất**. Thành phần chính thứ hai là tổ hợp tuyến tính **không tương quan** với thành phần thứ nhất và **giữ lại phương sai lớn nhất** trong phần còn lại của dữ liệu. Quá trình này được lặp lại cho đến khi thu được k thành phần chính, mỗi thành phần đều tối đa hóa phương sai còn lại và không tương quan tuyến tính với các thành phần trước đó.
- Dữ liệu có thể được biểu diễn bằng **một số lượng nhỏ các thành phần chính** ($k < p$), thay vì toàn bộ p biến ban đầu, nhưng vẫn giữ lại được phần lớn thông tin có ý nghĩa thống kê.

Input

- Một tập dữ liệu dạng ma trận $X \in \mathbb{R}^{n \times p}$, với n là số mẫu (quan sát), và p là số biến (đặc trưng).
- Giá trị $k \in \{1, 2, \dots, p\}$, là số lượng thành phần chính cần giữ lại, được lựa chọn sao cho đảm bảo **tối ưu hóa lượng thông tin giữ lại**.

Output

- Một tập k thành phần chính — các tổ hợp tuyến tính của các biến ban đầu, được sắp xếp theo thứ tự giảm dần về mặt phương sai.
- Ma trận dữ liệu mới $Z \in \mathbb{R}^{n \times k}$, thu được bằng cách chiếu dữ liệu ban đầu lên trục mới tạo bởi các thành phần chính.
- Tỷ lệ phương sai tích lũy được giữ lại bởi k thành phần đầu tiên, đóng vai trò như một thước đo định lượng cho hiệu quả giảm chiều.

1.5 Phương pháp phân tích thành phần chính

PCA có thể được triển khai dưới hai góc nhìn toán học như sau:

- Tiếp cận dựa trên **đại số tuyến tính** — dùng phân tích trị riêng (eigen-decomposition).

- Tiếp cận dựa trên **tối ưu hóa** — bài toán cực đại hóa phương sai có ràng buộc, giải bằng phương pháp nhân tử Lagrange.

1.5.1 Sử dụng đại số tuyến tính

Bài toán

Cho một tập dữ liệu gồm n quan sát, mỗi quan sát là một vector trong không gian $\mathbb{R}^p$. Tập dữ liệu được biểu diễn bởi ma trận:

$$X = \begin{bmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_n^T \end{bmatrix} \in \mathbb{R}^{n \times p}$$

với $x_i \in \mathbb{R}^p$ là vector đặc trưng của quan sát thứ i , và p là số biến (số chiều ban đầu). Ta giả sử dữ liệu đã được chuẩn hóa trung bình, tức là trung bình của mỗi cột (biến) là 0:

$$\frac{1}{n} \sum_{i=1}^n x_i = 0$$

Mục tiêu của PCA là tìm một hệ tọa độ mới gồm $k \leq p$ trục tọa độ sao cho:

- Mỗi trục là một tổ hợp tuyến tính của các biến ban đầu.
- Dữ liệu khi được chiếu lên các trục mới sẽ:
 - Không tương quan tuyến tính với nhau.
 - Giữ lại nhiều phương sai nhất có thể từ dữ liệu gốc.
 - Giảm chiều nhưng vẫn bảo toàn thông tin quan trọng.

Ma trận hiệp phương sai

Ma trận hiệp phương sai mẫu của dữ liệu chuẩn hóa được định nghĩa bởi:

$$\mathbf{S} = \frac{1}{n-1} \mathbf{X}^T \mathbf{X} \in \mathbb{R}^{p \times p}$$

Trong đó:

- $\mathbf{S}_{ij} = \text{Cov}(x^{(i)}, x^{(j)})$
- $\mathbf{S}$ là ma trận đối xứng nửa xác định dương.

Phân tích trị riêng (eigendecomposition)

Do $\mathbf{S}$ là ma trận đối xứng, theo định lý Spectral, tồn tại một tập p vector trực chuẩn $\{v_1, v_2, \dots, v_p\}$ và các giá trị riêng $\{\lambda_1, \lambda_2, \dots, \lambda_p\}$, sao cho:

$$\mathbf{S}v_j = \lambda_j v_j, \quad j = 1, \dots, p$$

với:

- $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p \geq 0$.
- $\{v_j\}$ tạo thành một cơ sở trực chuẩn của $\mathbb{R}^p$

Thành phần chính (principal components)

Thành phần chính thứ j được tính bởi:

$$\mathbf{z}_j = \mathbf{X}\mathbf{v}_j \in \mathbb{R}^n$$

Ma trận chứa tất cả các thành phần chính:

$$\mathbf{Z} = \mathbf{X}\mathbf{V}, \quad \text{với } \mathbf{V} = [\mathbf{v}_1, \dots, \mathbf{v}_p] \in \mathbb{R}^{p \times p}$$

Tính chất quan trọng:

$$\text{Cov}(\mathbf{Z}) = \mathbf{V}^\top \mathbf{S} \mathbf{V} = \Lambda$$

Trong đó Λ là ma trận đường chéo chứa các trị riêng λ_j . Điều này cho thấy các thành phần chính không tương quan với nhau, và:

$$\sum_{j=1}^p \text{Var}(\mathbf{z}_j) = \sum_{j=1}^p \lambda_j = \text{Trace}(\mathbf{S})$$

Giảm chiều dữ liệu

Để giảm chiều từ p xuống $k < p$, ta giữ lại k vector riêng ứng với k trị riêng lớn nhất:

$$\mathbf{V}_k = [\mathbf{v}_1, \dots, \mathbf{v}_k] \in \mathbb{R}^{p \times k}$$

Khi đó dữ liệu mới là:

$$\mathbf{Z}_k = \mathbf{X}\mathbf{V}_k \in \mathbb{R}^{n \times k}$$

Tỷ lệ phương sai được giữ lại:

$$\text{Variance Retained Ratio} = \frac{\sum_{j=1}^k \lambda_j}{\sum_{j=1}^p \lambda_j}$$

Giá trị k thường được chọn sao cho tỷ lệ này đạt ngưỡng mong muốn (chẳng hạn 90% hoặc 95%).

1.5.2 Tiếp cận tối ưu hóa Lagrange

Bài toán

Giả sử $\mathbf{x} \in \mathbb{R}^p$ là một vector ngẫu nhiên đã được chuẩn hóa trung bình (zero mean), và $\Sigma = \text{Cov}(\mathbf{x})$ là ma trận hiệp phương sai của $\mathbf{x}$:

$$\Sigma = \mathbb{E}[\mathbf{x}\mathbf{x}^\top]$$

Mục tiêu của PCA là tìm một tổ hợp tuyến tính $y = \alpha^\top \mathbf{x}$ sao cho y biểu diễn được nhiều thông tin nhất có thể từ $\mathbf{x}$. Vì độ phân tán (phương sai) là đại lượng đo thông tin trong thống kê, ta cần:

$$\text{Var}(y) = \alpha^\top \Sigma \alpha \rightarrow \max$$

với điều kiện chuẩn hóa $\|\alpha\|^2 = \alpha^\top \alpha = 1$ nhằm tránh nghiệm vô hướng không ràng buộc.

Xây dựng hàm Lagrange

Để cực đại hóa hàm mục tiêu có ràng buộc, ta sử dụng phương pháp nhân tử Lagrange. Xét hàm Lagrangian:

$$\mathcal{L}(\alpha, \lambda) = \alpha^\top \Sigma \alpha - \lambda(\alpha^\top \alpha - 1)$$

trong đó:

- $\alpha \in \mathbb{R}^p$ là vector tổ hợp tuyến tính cần tìm.

- $\Sigma \in \mathbb{R}^{p \times p}$ là ma trận hiệp phương sai (đối xứng, xác định dương/nửa xác định dương).
- λ là nhân tử Lagrange, sẽ trở thành trị riêng của Σ .

Điều kiện cực trị

Lấy đạo hàm riêng theo α và cho bằng 0:

$$\nabla_{\alpha} \mathcal{L} = 2\Sigma\alpha - 2\lambda\alpha = 0 \quad \Leftrightarrow \quad \Sigma\alpha = \lambda\alpha$$

Ta thấy rằng bài toán cực đại hóa phương sai tương đương với việc tìm các **vector riêng** α và **trị riêng** λ của ma trận Σ .

Ý nghĩa của nghiệm

Giá trị cực đại của phương sai là λ , với α là vector riêng tương ứng. Ta sắp xếp các trị riêng $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p$ và chọn α_1 là vector riêng ứng với λ_1 . Khi đó:

$$\text{Var}(y_1) = \alpha_1^T \Sigma \alpha_1 = \lambda_1 = \max_{\alpha: \|\alpha\|=1} \text{Var}(\alpha^T \mathbf{x})$$

Tương tự, các thành phần chính tiếp theo được xác định bằng cách:

- Tìm α_2 là vector riêng ứng với λ_2 , sao cho α_2 trực giao với α_1 .
- Tiếp tục như vậy để xây dựng hệ trực chuẩn $\{\alpha_1, \dots, \alpha_k\}$.

Mỗi tổ hợp tuyến tính $y_j = \alpha_j^T \mathbf{x}$ là một thành phần chính thứ j , với phương sai bằng λ_j .

Tóm lại, từ việc sử dụng tối ưu hóa Lagrange ta nhận thấy rằng:

- Việc tìm thành phần chính là quá trình cực đại hóa phương sai của tổ hợp tuyến tính.
- Bản chất của PCA là một bài toán trị riêng.
- Cung cấp trực giác rõ ràng: các phương sai được bảo toàn nằm ở các trị riêng lớn nhất của Σ .

Số chiều $k < p$ được chọn sao cho:

$$\frac{\sum_{j=1}^k \lambda_j}{\sum_{j=1}^p \lambda_j} \geq \text{ngưỡng mong muốn (ví dụ 90\%)}$$

1.5.3 Trục quan hóa

Giả sử về dữ liệu đầu vào Giả sử ta có một tập dữ liệu $X \in \mathbb{R}^{n \times 2}$, với n là số điểm dữ liệu. Mỗi hàng trong X là một điểm dữ liệu hai chiều, tức là:

$$X = \begin{bmatrix} x_1 & y_1 \\ x_2 & y_2 \\ \vdots & \vdots \\ x_n & y_n \end{bmatrix}$$

Mục tiêu của chúng ta là giảm dữ liệu từ 2D (x, y) xuống 1D trong khi giữ lại nhiều thông tin nhất có thể.

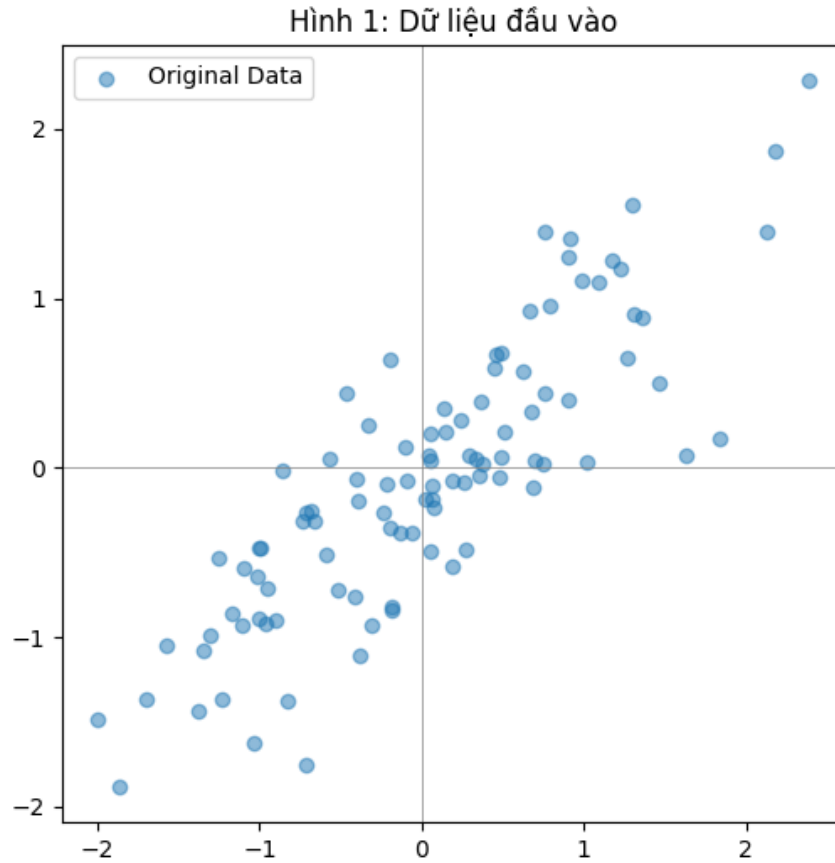


Figure 1: Giả sử về dữ liệu đầu vào

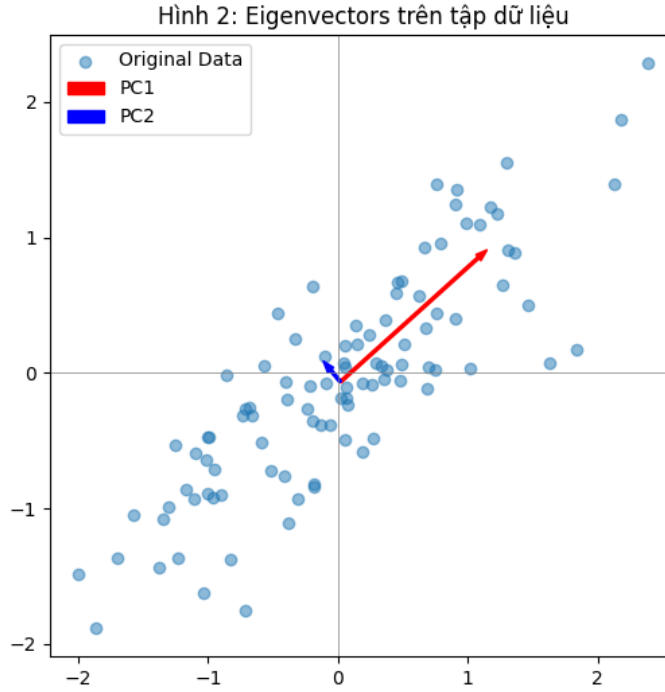


Figure 2: Eigenvectors đã được tính toán trên tập dữ liệu

Tìm phương hướng chính (Principal Component) Ta cần xác định phương hướng mà dữ liệu có phương sai lớn nhất. Điều này được thực hiện bằng cách tính ma trận hiệp phương sai:

$$\Sigma = \frac{1}{n} X^T X = \begin{bmatrix} \sigma_{xx} & \sigma_{xy} \\ \sigma_{xy} & \sigma_{yy} \end{bmatrix}$$

trong đó:

$$\sigma_{xx} = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2, \quad \sigma_{yy} = \frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2, \quad \sigma_{xy} = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$

Sau đó, ta tìm các trị riêng (eigenvalues) và vector riêng (eigenvectors) của ma trận Σ :

$$\Sigma \mathbf{v} = \lambda \mathbf{v}$$

- Eigenvector $\mathbf{v}_1$ ứng với eigenvalue lớn nhất λ_1 chính là phương hướng có phương sai lớn nhất — đây là trục PC1.

- Eigenvector $\mathbf{v}_2$ ứng với eigenvalue nhỏ hơn bị bỏ qua trong quá trình giảm chiều.

Chiếu dữ liệu lên trục chính PC1 Mỗi điểm dữ liệu $\mathbf{x}_i = [x_i, y_i]$ được chiếu vuông góc lên PC1 bằng cách tính:

$$z_i = \mathbf{x}_i \cdot \mathbf{v}_1$$

Khi đó, tập dữ liệu sau khi chiếu xuống 1D trở thành:

$$\mathbf{Z} = \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_n \end{bmatrix}$$

Đây là tập dữ liệu đã giảm chiều.

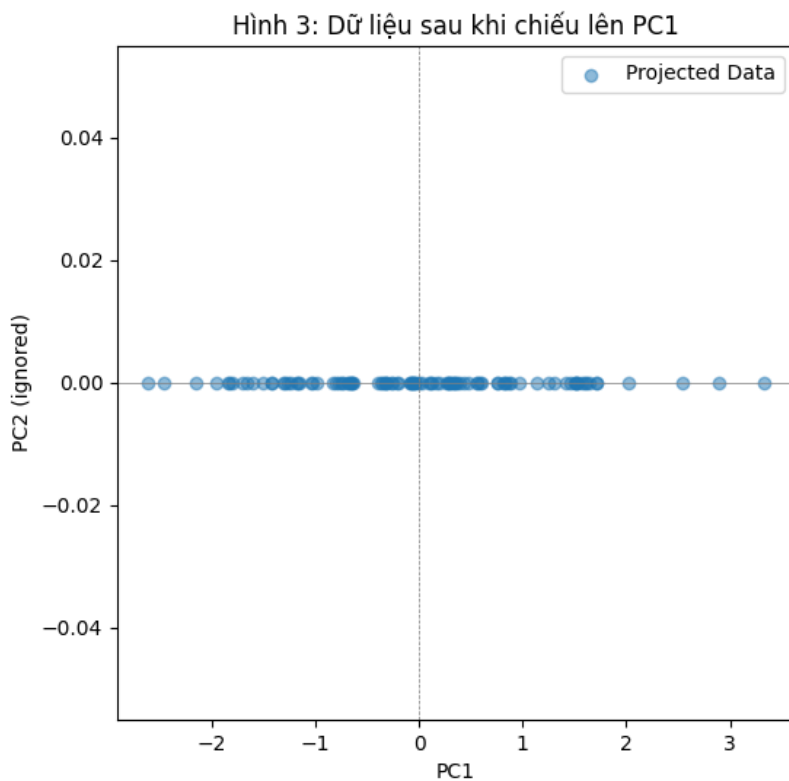


Figure 3: Dữ liệu sau khi chiếu lên trục chính PC1

Dữ liệu sau khi giảm chiều Sau khi giảm chiều, dữ liệu chỉ còn một giá trị z_i duy nhất cho mỗi điểm thay vì hai giá trị (x_i, y_i) . Dữ liệu giờ đây nằm trên một đường thẳng, tức là ta đã giảm từ không gian 2D xuống không gian 1D.

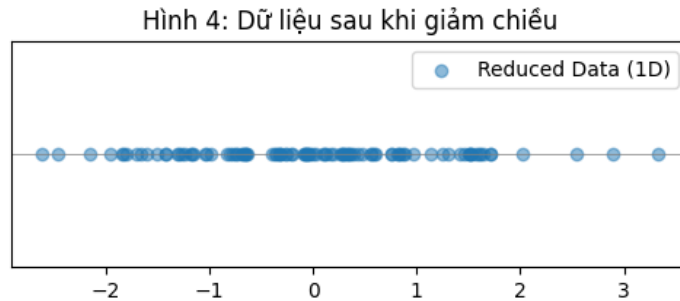


Figure 4: Dữ liệu sau khi giảm chiều từ 2D xuống 1D

1.6 Giải thuật

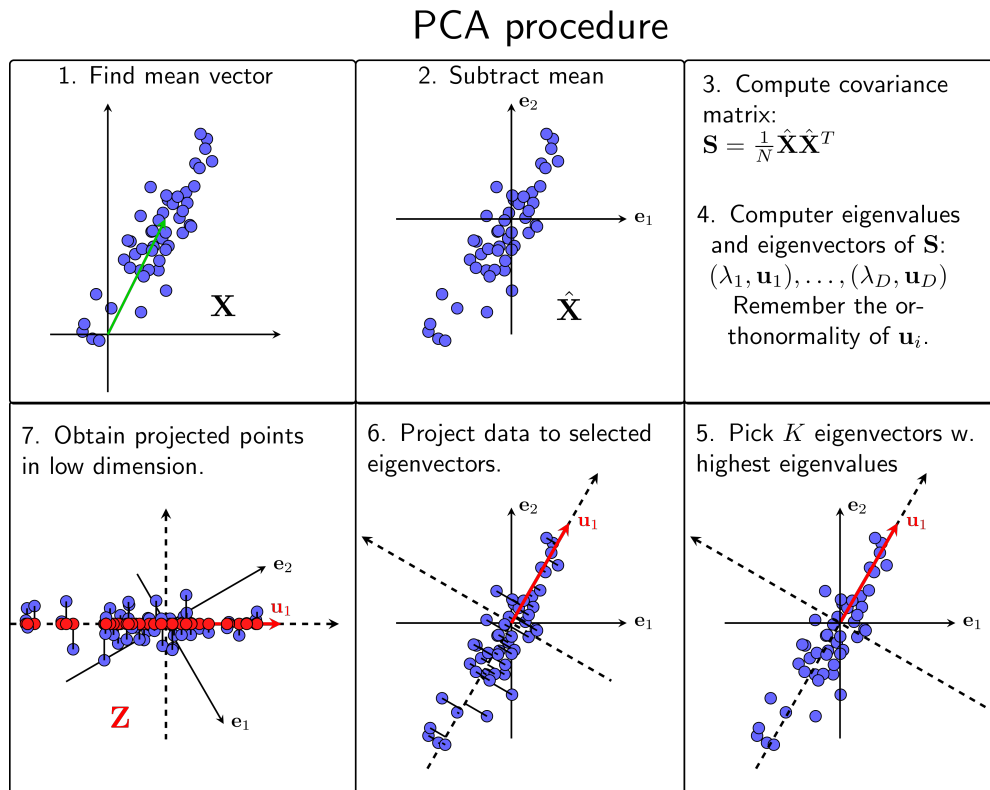


Figure 5: Giải thuật Phân tích thành phần chính

Tính toán phương sai và phương hướng dữ liệu

- **Tính toán trung bình của dữ liệu:** với x_{ij} là giá trị của biến thứ j ở mẫu thứ i . Giá trị trung bình giúp xác định tâm của dữ liệu, hỗ trợ trong việc chuẩn hóa.
- **Tính toán phương sai:** đo lường mức độ phân tán của dữ liệu xung quanh giá trị trung bình, giúp PCA xác định các hướng có độ biến thiên cao nhất.
- **Xác định hướng chính:** Xác định phương hướng có phương sai lớn nhất. Hướng này là chiều dữ liệu thay đổi nhiều nhất, từ đó giúp PCA trích xuất các đặc trưng quan trọng nhất.

Tính toán vector riêng và giá trị riêng

- **Tính ma trận hiệp phương sai:** với X là ma trận dữ liệu sau khi chuẩn hóa. Ma trận hiệp phương sai giúp biểu diễn mối quan hệ giữa các biến trong dữ liệu.
- **Tính toán giá trị riêng và vector riêng:** Giải phương trình đặc trưng $S\vec{v} = \lambda\vec{v}$, trong đó λ là giá trị riêng và $\vec{v}$ là vector riêng. Giá trị riêng đại diện cho lượng thông tin (phương sai) mà một hướng chính (vector riêng) mang lại.
- **Chọn vector riêng chính:** Chọn vector riêng có giá trị riêng lớn nhất để tạo không gian mới. Những vector riêng này xác định các chiều dữ liệu quan trọng nhất.

Chiếu dữ liệu lên trục thành phần chính

- **Biến đổi dữ liệu gốc:** Chiếu dữ liệu vào không gian thành phần chính $Z = XW$, với W là ma trận chứa vector riêng chính.
- **Tạo không gian dữ liệu mới:** Hình thành tập dữ liệu mới có số chiều thấp hơn nhưng vẫn giữ lại phần lớn thông tin cần thiết.

1.7 Đóng góp

Báo cáo này nhằm mục đích:

- Trình bày cơ sở lý thuyết và ứng dụng của PCA trong nhiều lĩnh vực.
- Phân tích và so sánh các phương pháp lựa chọn số thành phần chính tối ưu.
- Cung cấp hình ảnh minh họa trực quan để hỗ trợ việc hiểu và triển khai PCA trong thực tế.

- Đánh giá hiệu suất của PCA khi áp dụng vào các tập dữ liệu thực tế.

1.8 Thách thức

Mặc dù PCA là một phương pháp mạnh mẽ, nhưng nó vẫn gặp phải một số thách thức:

- **Xử lý dữ liệu phi tuyến:** PCA chỉ hoạt động tốt với dữ liệu tuyến tính, trong khi dữ liệu thực tế thường có cấu trúc phi tuyến.
- **Chọn số lượng thành phần chính tối ưu:** Nếu chọn quá ít, thông tin có thể bị mất; nếu chọn quá nhiều, mô hình có thể không giảm được độ phức tạp.
- **Chỉ giữ lại thành phần có thay đổi phương sai lớn nhất:** PCA chỉ đảm bảo giữ lại các chiều dữ liệu có sự thay đổi lớn nhất trong dữ liệu gốc, không đảm bảo rằng các thành phần chính được giữ lại có tác động lớn tới tổng thể trong trường hợp nhất định (như cần phân loại dữ liệu sẽ hiệu quả hơn khi sử dụng LDA, ICA).
- **Tính toán trên dữ liệu lớn:** PCA yêu cầu tính toán ma trận hiệp phương sai và giải bài toán trị riêng, có thể trở nên rất tốn tài nguyên với dữ liệu lớn.

2 Các công trình nghiên cứu liên quan

Trong quá trình phát triển của lĩnh vực học máy và nhận dạng mẫu, PCA đã được mở rộng và cải tiến qua nhiều biến thể để khắc phục các hạn chế của phiên bản gốc, cũng như thích nghi với các bài toán cụ thể hơn. Phần này trình bày các công trình tiêu biểu đã mở rộng hoặc điều chỉnh PCA nhằm tăng khả năng biểu diễn, tăng độ chính xác trong nhận dạng hoặc xử lý dữ liệu lớn. Mỗi công trình được giới thiệu ngắn gọn về nguồn gốc, ý tưởng chính và sau đó được hệ thống hóa qua bảng tổng hợp để tiện so sánh.

2.1 Tổng quan các biến thể PCA

Eigenfaces (Turk & Pentland, 1991)

Công trình này đánh dấu bước khởi đầu cho việc áp dụng PCA trong nhận dạng khuôn mặt. Ý tưởng cốt lõi là sử dụng PCA để trích xuất những hướng phương sai chính trong ảnh khuôn mặt, tạo nên không gian đặc trưng gọi là “eigenfaces”. Các ảnh mới được chiếu vào không gian này để nhận diện.

Kernel PCA (Schölkopf et al., 1997)

Được phát triển nhằm mở rộng PCA sang miền phi tuyến, Kernel PCA sử dụng “kernel trick” để ánh xạ dữ liệu vào không gian đặc trưng có chiều cao, từ đó áp dụng PCA tuyến tính. Nhờ vậy, các cấu trúc phi tuyến trong dữ liệu gốc có thể được khai thác hiệu quả hơn.

Robust PCA (Candes et al., 2011)

Ra đời trong bối cảnh dữ liệu thực có nhiều nhiễu và outlier, Robust PCA xem dữ liệu là tổng của ma trận thấp hạng (dữ liệu thật) và ma trận thưa (nhiễu). Bài toán được giải qua tối ưu lồi nhằm tách biệt phần thông tin chính xác và phần sai lệch.

Incremental PCA (Hall et al., 1998; Weng et al., 2003)

Nhằm phục vụ bài toán học trực tuyến và xử lý luồng dữ liệu lớn, Incremental PCA cho phép cập nhật kết quả PCA từng bước theo thời gian, mà không cần tính lại từ đầu toàn bộ.

2.2 So sánh các phương pháp

Phương pháp	Ý tưởng chính	Ưu điểm	Nhược điểm	Hiệu suất thực nghiệm	Ứng dụng nổi bật
Eigenfaces	Dùng PCA để biểu diễn khuôn mặt bằng các thành phần chính (eigenfaces)	Đơn giản, tốc độ cao, dễ triển khai	Nhạy cảm với nhiễu, điều kiện ánh sáng	Đạt ~88% trên bộ ORL (5 ảnh train/test)	Nhận diện khuôn mặt
Kernel PCA	Sử dụng kernel để thực hiện PCA trong không gian phi tuyến	Mô hình hóa được dữ liệu phi tuyến, chính xác cao	Tính toán tốn kém, khó khôi phục ảnh gốc	Đạt ~98% trên bộ AT&T với kernel đa thức	Nhận diện khuôn mặt, phân loại ảnh
Robust PCA	Phân tách dữ liệu thành phần thấp hạng và nhiễu thừa bằng tối ưu lồi	Chịu được nhiễu mạnh, cơ sở toán học vững chắc	Tính toán nặng, nhạy với tham số	Giảm MSE tái tạo ảnh nhiễu từ 19.35 xuống 16.54	Giám sát video, khử nhiễu ảnh
Incremental PCA	Cập nhật PCA từng bước theo thời gian khi có dữ liệu mới	Phù hợp học on-line, nhanh, tiết kiệm bộ nhớ	Khó cài đặt, sai số tích lũy, cần cơ chế “quên”	Sai khác <1% so với PCA đầy đủ	Xử lý luồng video, học máy trực tuyến

Table 1: So sánh các biến thể PCA tiêu biểu trong nghiên cứu

3 Phương pháp

Nhận thấy sự hạn chế của PCA trong việc xử lý các dữ liệu phi tuyến, đồ án tập trung cài đặt và phân tích **Kernel PCA** - một biến thể của PCA tập trung vào phân tích PCA trên không gian đặc trưng, giúp giải quyết vấn đề lớn nhất mà PCA đang gặp phải.

Kernel PCA là một mở rộng của PCA tuyến tính sang không gian đặc trưng phi tuyến, cho phép biểu diễn các cấu trúc **cong, xoắn, hoặc phức tạp** trong dữ liệu bằng một biến đổi tuyến tính ở **không gian khác**.

Để thực hiện điều này, ta ánh xạ dữ liệu vào một không gian khác thông qua kernel, sau đó tiến hành PCA tại đó mà không cần biểu diễn tường minh.

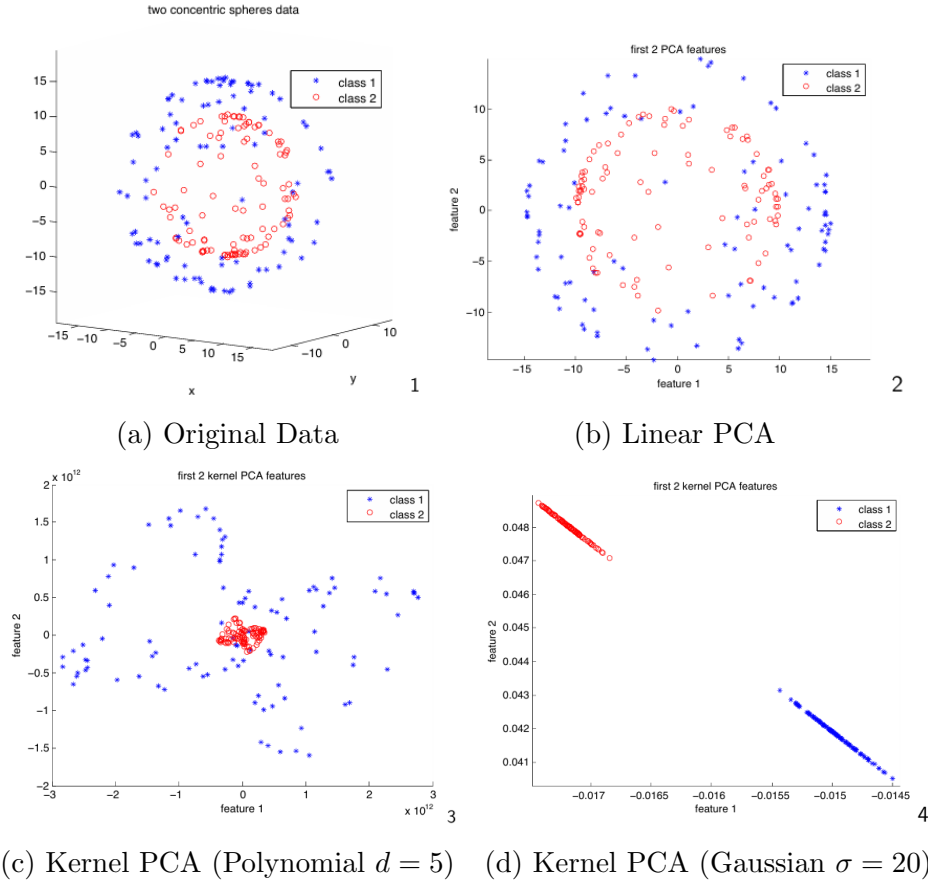


Figure 6: Trực quan hóa giảm chiều dữ liệu bằng các phương pháp khác nhau

3.1 Mục tiêu

Tìm các thành phần chính (principal components) **trong không gian đặc trưng**, tương ứng với các chiều biến thiên lớn nhất của dữ liệu - giúp giảm chiều mà vẫn giữ lại thông tin quan trọng.

3.2 Phương pháp thực hiện

3.2.1 Ánh xạ dữ liệu vào không gian đặc trưng

Dữ liệu đầu vào:

$$x_1, x_2, \dots, x_n \in \mathbb{R}^p$$

trong đó n là số mẫu quan sát và p là số biến.

Ta giả định có một ánh xạ phi tuyến:

$$\phi: \mathbb{R}^p \rightarrow \mathcal{F}$$

trong đó $\mathcal{F}$ là một không gian vector có chiều rất cao (không gian đặc trưng), thậm chí vô hạn chiều. Mục đích của việc ánh xạ này là để làm tuyến tính hóa các mối quan hệ phi tuyến trong dữ liệu.

Tuy nhiên ta không thể tính trực tiếp $\phi(x)$ (vì không biết cụ thể hàm ánh xạ) mà quan tâm tới **tích vô hướng** $\langle \phi(x_i), \phi(x_j) \rangle$ (kernel trick), gọi là kernel:

$$k(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle$$

vì không gian có thể vô hạn chiều, nên tính toán trực tiếp là rất khó khăn, việc sử dụng tích vô hướng có thể được tính dễ dàng nhờ các hàm như:

- Polynomial:

$$k(x, x') = (\langle x, x' \rangle + c)^d$$

- Gaussian:

$$k(x, x') = \exp\left(-\frac{\|x - x'\|^2}{2\sigma^2}\right)$$

3.2.2 Tính ma trận kernel

Ta xây dựng ma trận kernel

$$K \in \mathbb{R}^{n \times n}, \quad K_{ij} = k(x_i, x_j)$$

chứa mọi tích vô hướng giữa các điểm trong không gian đặc trưng $\mathcal{F}$, thay thế cho ma trận hiệp phương sai trong PCA thông thường.

3.2.3 Tâm hóa dữ liệu trong không gian đặc trưng

Trong PCA tuyến tính, dữ liệu cần được trung bình hóa (mean zero), tương tự ta thực hiện trong không gian đặc trưng:

$$\frac{1}{n} \sum_{i=1}^n \phi(x_i) = 0$$

nhưng không thể tính trung bình trong không gian $\mathcal{F}$, nên ta tâm hóa ma trận kernel. Ta định nghĩa:

$$\tilde{\phi}(x_i) = \phi(x_i) - \mu$$

Khi đó tích vô hướng trong không gian đặc trưng:

$$\langle \tilde{\phi}(x_i), \tilde{\phi}(x_j) \rangle = \langle \phi(x_i) - \mu, \phi(x_j) - \mu \rangle = \langle \phi(x_i), \phi(x_j) \rangle - \langle \phi(x_i), \mu \rangle - \langle \mu, \phi(x_j) \rangle + \langle \mu, \mu \rangle$$

Viết lại bằng kernel

$$\tilde{K} = K - 1_n K - K 1_n + 1_n K 1_n$$

với mỗi giá trị

$$\tilde{K}_{ij} = K_{ij} - \frac{1}{n} \sum_{l=1}^n K_{il} - \frac{1}{n} \sum_{l=1}^n K_{lj} + \frac{1}{n^2} \sum_{l=1}^n \sum_{m=1}^n K_{lm}$$

tức là loại bỏ trung bình để đảm bảo dữ liệu đã được tâm hóa, trong đó $1_n K$ là trung bình dòng (trung bình của mỗi điểm so với tất cả), $K 1_n$ là trung bình cột còn $1_n K 1_n$ là trung bình của toàn bộ ma trận.

Tức vì ta không thể "dời" từng $\phi(x_i)$ vì không thể tính toán được, nhưng ta lại có tích vô hướng giữa các điểm, nên ta có thể dời chúng về trung tâm mà không cần truy cập tọa độ thật.

3.2.4 Giải bài toán trị riêng

Sau khi có $\tilde{K}$, ta giải được trị riêng:

$$\tilde{K}\alpha = \lambda\alpha$$

thay cho PCA thông thường

$$Cv = \lambda v, \quad C = \frac{1}{n}X^T X$$

tức thay X bằng kernel (đại diện cho $\phi(x_i)$).

3.2.5 Chuẩn hóa các vector riêng

Để đảm bảo các vector riêng tương ứng với các thành phần chính có độ dài chuẩn:

$$\lambda_j \alpha_j^T \alpha_j = 1 \Rightarrow \alpha_j \leftarrow \frac{\alpha_j}{\sqrt{\lambda_j}}$$

với mục tiêu đảm bảo độ dài của các thành phần chính trong không gian đặc trưng là 1 (điều kiện chuẩn hóa).

Khi đó, vector chính thứ j trong không gian đặc trưng là

$$v_j = \sum_{i=1}^n \alpha_i^{(j)} \phi(x_i)$$

đảm bảo các trục tọa độ là cơ sở trực chuẩn.

3.2.6 Chiếu dữ liệu vào không gian mới

Với mỗi điểm x , tọa độ theo thành phần chính thứ j là:

$$z_j(x) = \sum_{i=1}^n \alpha_i^{(j)} k(x_i, x)$$

tức là không cần ánh xạ $\phi(x)$ mà chỉ cần biết kernel giữa x và các điểm huấn luyện. Khi đó biểu diễn mới $z_j(x)$ sẽ là tọa độ của x trên trục thứ j , biến đổi toàn bộ dữ liệu thành ma trận $Z \in \mathbb{R}^{n \times k}$

4 Cài đặt và kết quả thực nghiệm

4.1 Mô tả Bộ dữ liệu (Datasets)

Trong phần thực nghiệm, chúng tôi sử dụng hai bộ dữ liệu ảnh khuôn mặt chuẩn:

4.1.1 Olivetti Faces

- Chứa 400 ảnh grayscale kích thước 64x64 pixels.
- Thuộc về 40 người khác nhau, mỗi người có 10 ảnh.
- Các ảnh có sự thay đổi về ánh sáng, biểu cảm và một chút về góc chụp.



Figure 7: Ảnh mẫu Olivetti (4 người đầu)

4.1.2 LFW (Labeled Faces in the Wild) - Subset

- Bộ dữ liệu lớn chứa ảnh khuôn mặt được thu thập từ internet, thể hiện điều kiện "ngoài đời thực" với nhiều biến thể lớn về góc mặt, ánh sáng, biểu cảm, nền, độ phân giải, và vật che khuất.
- Do kích thước lớn, chúng tôi sử dụng một tập con bằng cách chỉ lấy những người có ít nhất 70 ảnh (`min_faces_per_person=70`).

- Tập con này bao gồm 1288 ảnh thuộc về 7 người khác nhau.
- Ảnh được sử dụng ở kích thước gốc sau khi tải về (khoảng 62x47 pixels, 2914 chiều) và được chuyển sang ảnh xám (grayscale).



Figure 8: Ảnh mẫu LFW

4.2 Giải thuật và Code Tương ứng

Phần cài đặt tập trung vào việc so sánh PCA (tự code bằng SVD) và KPCA (sử dụng lớp tự định nghĩa với kernel RBF) trên hai bộ dữ liệu.

4.2.1 Tiền xử lý

Chuẩn hóa dữ liệu: Sử dụng `StandardScaler` để đưa các đặc trưng pixel về cùng thang đo (trung bình 0, độ lệch chuẩn 1). Đây là bước quan trọng cho cả PCA và KPCA (đặc biệt với RBF kernel).

```
1 # 4. Standardize the ENTIRE selected dataset
2 print("\n--- Standardizing data ---")
3 scaler = StandardScaler()
```



```
4 X_scaled = scaler.fit_transform(X)
```

Listing 1: Data standardization using StandardScaler

4.2.2 PCA Tuyến tính (Cài đặt bằng SVD)

Giải thuật:

- Nhận đầu vào là dữ liệu đã chuẩn hóa X_{scaled} (kích thước $n \times D$).
- Thực hiện Phân tích giá trị suy biến (SVD): $X_{\text{scaled}} = U \cdot S \cdot V^T$.
- Scores (tọa độ mới) trong không gian d chiều được tính bằng $X_{\text{pca}} = U[:, :d] \cdot S[:, d]$ (hoặc $U[:, :d] @ \text{diag}(S[:, d])$).

```
1 def manual_pca(X_scaled, n_components):
2     """
3     Perform PCA using SVD on standardized data (zero mean).
4     Input:
5         X_scaled: Standardized data (samples x features)
6         n_components: Desired number of dimensions (d)
7     Output:
8         X_pca: Data projected down to d dimensions (scores)
9     """
10    print(f"--- Performing SVD for PCA (d={n_components})...")
11    U, s, Vt = np.linalg.svd(X_scaled, full_matrices=False)
12    print(f"--- Calculating scores for PCA...")
13    X_pca = U[:, :n_components] * s[:n_components]
14    return X_pca
```

Listing 2: Function to perform PCA using SVD

4.2.3 Kernel PCA (KPCA)

Giải thuật (Tóm tắt các bước chính đã cài đặt):

- Tính Ma trận Kernel (RBF): Dùng hàm kernel RBF $k(x, y) = \exp(-\gamma \cdot \|x - y\|^2)$ để tính ma trận K (nxn) từ dữ liệu X_{scaled} .

- Tâm hóa Kernel: Biến đổi K thành K_{centered} bằng công thức tâm hóa.
- Phân tích trị riêng: Tìm trị riêng λ và vector riêng v của K_{centered} bằng `np.linalg.eig`.
- Chuẩn hóa & Sắp xếp: Tính giá trị suy biến $\sigma = \sqrt{\lambda}$ và vector riêng chuẩn hóa $\alpha = v/\sqrt{\lambda}$.
- Sắp xếp các cặp (σ, α) theo σ giảm dần.
- Chiều (Tính Scores): Chọn d thành phần đầu.
- Tạo ma trận đường chéo Σ_d (từ σ) và ma trận A_d (các cột là α).
- Tính Scores = $\Sigma_d @ A_d^T$.

Code (Các thành phần chính trong `kernels.py` và `kpca.py`):

`kernels.py` (Hàm RBF):

```

1 def rbf(self, x, y):
2     """
3     k(x, y) = exp(- gamma * ||x-y||^2)
4     Hiperparametros: gamma
5     """
6     return np.exp(-self.gamma * (np.linalg.norm(x-y)**2))

```

Listing 3: Hàm tính kernel RBF

`kpca.py` (Các hàm chính thực hiện giải thuật)

```

1 def __init__(self, X, kernel, d):
2     """
3     KPCA object
4     Parameters
5     -----
6     X: dxn matrix
7     kernel: kernel function from kernel class
8     d: number of principal components to be chosen
9     """
10    self.X = X
11    self.kernel = kernel
12    self.d = d
13
14    def is_pos_semidef(self, K):
15        return np.all(K >= 0) # Incorrect check, eigvals needed. Placeholder.
16
17    def kernel_matrix(self):
18        """
19        Compute kernel matrix
20        Output:
21        K: nxn matrix
22        """

```

```

23     n, f = self.X.shape
24     K_aux = []
25     for fil in range(n):
26         K_aux_row = []
27         for col in range(n):
28             # Assuming kernel function takes two vectors
29             K_aux_row.append(self.kernel(self.X[fil, :], self.X[col, :]))
30         K_aux.append(K_aux_row)
31     K = np.array(K_aux)
32
33     # Centering K
34     ones = np.ones(K.shape)/n
35     K = K - ones@K - K@ones + ones@K@ones
36     return K
37
38 def descomp(self):
39     """
40     Decomposition of K
41     Output:
42     tuplas_eig: List of ordered tuples by singular
43                 values; (singular_value, eigenvector)
44     """
45     self.K = self.kernel_matrix()
46     eigval, eigvec = np.linalg.eig(self.K)
47
48     # Check if K is positive semidefinite (optional, for numerical stability)
49     # if not self.is_pos_semidef(self.K):
50     #     warnings.warn("Kernel matrix K is not semidefinite positive")
51
52     # Normalize eigenvectors and compute singular values of K
53     # Ensure eigenvalues are non-negative before sqrt
54     eigval[eigval < 1e-10] = 0 # Threshold small negative values due to numerical errors
55     tuplas_eig = [(np.sqrt(eigval[i]), eigvec[:, i]/np.sqrt(eigval[i]) if eigval[i] > 1e-10 else np.zeros_like(eigvec[:,i])) for
i in range(len(eigval))]
56
57     # Sort eigenvalues and eigenvectors
58     tuplas_eig.sort(key=lambda x: x[0], reverse=True)
59     return tuplas_eig
60
61 def project(self):
62     """
63     Compute scores
64     Output:
65     Scores: T = sigma * V_d^t
66     """
67     tuplas_eig = self.descomp()
68
69     # Select the first d components
70     tuplas_eig_dim = tuplas_eig[:self.d]
71     self.sigma = np.diag([tuplas_eig_dim[i][0] for i in range(self.d)])
72     self.alfas = np.array([tuplas_eig_dim[i][1] for i in range(self.d)]).T
73     # Projection (Scores): T = K_centered @ Alphas -- or simpler T = Sigma @ Alphas^T if Alphas are normalized eigenvectors of K
74     # Scores = self.K @ self.alfas # Projection onto principal components in feature space mapped back?
75     # Or simply the scaled eigenvectors (principal components in the transformed space)
76     # Let's assume Scores are calculated as in the formula provided: Sigma @ Alpha^T
77     # Note: The formula T = Sigma * V^T is for PCA from SVD(X), not KPCA from EIG(K)
78     # For KPCA, scores are often the projection: K_centered @ Alphas or simply Alphas scaled by sqrt(eigenvalues)
79     # The formula T = Sigma @ A^T implies A is nxn and T is dxn. Let's stick to the provided formula in the doc.
80     Scores = self.sigma @ self.alfas.T # This yields d x n scores
81     self.scores = Scores.T # Convert to n x d scores
82     return self.scores

```

Listing 4: Lớp KPCA và các phương thức chính

4.3 Giải thích Độ đo (k-NN CV Accuracy)

Mục đích: Vì trong bài toán nhận diện khuôn mặt thường giảm chiều dữ liệu bằng PCA/KPCA về 100->200 hoặc hơn, nên chúng em không thể dùng cách trực quan hóa để đánh giá hiệu quả của PCA/KPCA được. Do đó, chúng em cần một phương pháp định lượng để đo lường mức độ hiệu quả của các đặc trưng này trong việc phân biệt các khuôn mặt khác nhau. Bộ phân loại k-NN kết hợp với Cross-Validation được sử dụng làm độ đo cho mục đích này.

4.3.1 Bộ phân loại k-NN (k-Nearest Neighbors - k-Láng giềng Gần nhất)

Ý tưởng chính: Để xác định một khuôn mặt mới thuộc về ai, hãy xem những khuôn mặt (đã biết) trông giống nó nhất (gần nó nhất trong không gian đặc trưng). Khuôn mặt mới có khả năng thuộc về cùng nhóm với đa số những "hàng xóm" gần nhất đó.

Cách hoạt động:

- Input: Một vector đặc trưng d chiều của ảnh cần phân loại (X_{new}), toàn bộ dữ liệu huấn luyện đã giảm chiều (X_{train} - gồm nhiều vector d chiều) và nhãn tương ứng (y_{train} - ID người), số k (ví dụ: $k = 5$).
- Tính khoảng cách: Tính khoảng cách Euclidean giữa X_{new} và *từng* vector trong X_{train} .
- Tìm k láng giềng: Chọn ra k vector trong X_{train} có khoảng cách ngắn nhất tới X_{new} .
- Bỏ phiếu: Xem nhãn (ID người) của k láng giềng này.
- Nhãn nào xuất hiện nhiều lần nhất sẽ là kết quả dự đoán cho X_{new} .
- Trong code: `knn = KNeighborsClassifier(n_neighbors=5)` chính là tạo ra bộ phân loại này với $k = 5$.

4.3.2 Kỹ thuật Cross-Validation (CV - Đánh giá chéo)

Vấn đề: Nếu chúng ta dùng *toàn bộ* dữ liệu để học rồi lại dùng chính dữ liệu đó để đánh giá xem k-NN đoán đúng bao nhiêu %, thì kết quả sẽ quá lạc quan, không phản ánh đúng khả năng của mô hình khi gặp ảnh *mới*.

Giải pháp: Cross-Validation chia dữ liệu ra làm nhiều phần, lần lượt dùng một phần để "giả làm dữ liệu mới" (tập kiểm tra/validation) và các phần còn lại để "huấn luyện". Điều này cho phép đánh giá mô hình trên nhiều tập con khác nhau của dữ liệu.

Cách hoạt động (5-Fold CV - `cv=5` trong code):

- Chia: Dữ liệu (ví dụ 1288 ảnh LFW và nhãn y) được xáo trộn và chia ngẫu nhiên thành 5 "Fold" (phần) bằng nhau.
- Lặp 5 lần:
 - Lần 1: Dùng Fold 1 làm tập kiểm tra, huấn luyện k-NN trên Fold 2, 3, 4, 5. Tính độ chính xác trên Fold 1.
 - Lần 2: Dùng Fold 2 làm tập kiểm tra, huấn luyện k-NN trên Fold 1, 3, 4, 5. Tính độ chính xác trên Fold 2.
 - ...
 - Lần 5: Dùng Fold 5 làm tập kiểm tra, huấn luyện k-NN trên Fold 1, 2, 3, 4. Tính độ chính xác trên Fold 5.
- Trong code: `cross_val_score(knn, X_pca, y, cv=5)` thực hiện toàn bộ quy trình 5-Fold CV này và trả về một mảng chứa 5 giá trị độ chính xác của 5 lần kiểm tra.

4.3.3 Độ chính xác k-NN CV Trung bình

Tính toán: Là giá trị trung bình cộng của các độ chính xác thu được từ các Fold trong Cross-Validation (ví dụ: trung bình của 5 giá trị accuracy ở trên).

Ý nghĩa: Con số này cho chúng ta một ước lượng đáng tin cậy về việc: "Nếu tôi dùng các đặc trưng do PCA/KPCA tạo ra và một bộ phân loại k-NN đơn giản, thì khả năng hệ thống nhận diện đúng một khuôn mặt mới là khoảng bao nhiêu phần trăm?".

Sử dụng để so sánh: Bằng cách tính độ đo này cho cả kết quả của PCA và KPCA (với các tham số khác nhau), ta có thể so sánh xem phương pháp/tham số nào tạo ra các đặc trưng giúp k-NN phân tách và nhận diện các người khác nhau tốt hơn. Độ chính xác k-NN CV càng cao, chứng tỏ đặc trưng càng có tính phân biệt tốt.

4.4 Trình bày Kết quả và Nhận xét

Bảng kết quả tóm tắt độ chính xác nhận diện khuôn mặt (sử dụng bộ phân loại k-NN với 5-fold Cross-Validation) sau khi áp dụng PCA và KPCA (kernel RBF với gamma cố định) để giảm chiều dữ liệu xuống các số chiều d khác nhau trên hai bộ dữ liệu: Olivetti và LFW.

--- Olivetti (40 nguoi, KPCA g=3.0E-07) ---		
Số chiều (d)	PCA Acc k-NN	KPCA Acc k-NN

50	0.8400	0.8800
100	0.8375	0.7250
150	0.8225	0.5150
200	0.8225	0.3700
250	0.8175	0.2675

Olivetti - Số chiều tốt nhất (k-NN Accuracy):		
- PCA: d=50 (Acc: 0.8400)		
- KPCA (RBF): d=50 (Acc: 0.8800)		
=====		
--- LFW (7 nguoi, KPCA g=3.0E-07) ---		
Số chiều (d)	PCA Acc k-NN	KPCA Acc k-NN

50	0.5823	0.7399
100	0.6002	0.7018
150	0.6040	0.6576
200	0.6017	0.6203
250	0.5986	0.5815

LFW - Số chiều tốt nhất (k-NN Accuracy):		
- PCA: d=150 (Acc: 0.6040)		
- KPCA (RBF): d=50 (Acc: 0.7399)		
=====		

Figure 9: Bảng tóm tắt kết quả Accuracy k-NN (5-Fold CV)

Nhận xét:

- **KPCA cho thấy ưu thế (khi có tham số phù hợp):**

- Trên cả hai bộ dữ liệu, KPCA (với kernel RBF và giá trị γ được chọn) đều đạt được độ chính xác k-NN cao hơn so với PCA tại cấu hình tốt nhất của mỗi phương pháp.

- Olivetti: KPCA ($d=50$, $g=3.0E-07$) đạt 88.0%, cao hơn PCA ($d=50$) là 84.0% (cải thiện 4
- LFW: KPCA ($d=50$, $g=3.0E-07$) đạt 74.0%, cao hơn đáng kể so với PCA tốt nhất ($d=150$) chỉ đạt 60.4% (cải thiện 13.6%).
- Kết luận: Điều này cho thấy khả năng xử lý các mối quan hệ phi tuyến của KPCA thực sự mang lại lợi ích trong việc tạo ra các đặc trưng có tính phân biệt cao hơn cho bài toán nhận diện khuôn mặt trên cả hai bộ dữ liệu này. Lợi thế càng rõ rệt trên bộ LFW phức tạp hơn.

- **Số chiều tối ưu (d) là quan trọng và khác biệt:**

- KPCA: Đạt hiệu năng cao nhất ở số chiều tương đối thấp ($d=50$) trên cả hai bộ dữ liệu. Khi tăng d lên cao hơn, độ chính xác của KPCA giảm đi đáng kể. Điều này gợi ý rằng các thông tin phi tuyến quan trọng nhất được KPCA (với kernel RBF và gamma này) nắm bắt nằm chủ yếu ở các thành phần đầu, việc thêm các thành phần sau có thể gây nhiễu.
- PCA: Số chiều tối ưu của PCA có xu hướng cao hơn ($d=50$ trên Olivetti nhưng $d=150$ trên LFW). Hiệu năng của PCA cũng ít bị ảnh hưởng tiêu cực hơn KPCA khi tăng d .
- Kết luận: Không phải cứ giảm chiều càng nhiều (giữ lại d càng lớn) là càng tốt. Cần tìm ra số chiều tối ưu cho từng phương pháp và từng bộ dữ liệu. KPCA dường như cô đọng thông tin hữu ích vào ít chiều hơn trong các thử nghiệm này.

- **Ảnh hưởng của Dataset:**

- Độ chính xác trên Olivetti (88%) cao hơn nhiều so với trên LFW (74%).
- Điều này phản ánh đúng tính chất: Olivetti là bộ dữ liệu "sạch" và dễ hơn, trong khi LFW "in the wild" phức tạp và khó hơn nhiều.

Kết luận: Các thử nghiệm so sánh cho thấy Kernel PCA (sử dụng kernel RBF với tham số gamma phù hợp) có khả năng vượt trội hơn PCA tuyến tính trong bài toán trích xuất đặc trưng cho nhận diện khuôn mặt trên cả bộ dữ liệu Olivetti và LFW. KPCA đạt được độ chính xác phân loại k-NN cao hơn, đặc biệt rõ rệt trên bộ LFW phức tạp hơn. Kết quả cũng nhấn mạnh tầm quan trọng của việc lựa chọn đúng số chiều cần giữ lại (d), với KPCA thường đạt hiệu quả tối ưu ở số chiều thấp

hơn so với PCA trong các thử nghiệm này. Điều này chứng minh tiềm năng của các phương pháp phi tuyến như KPCA trong việc xử lý các biến thể phức tạp của dữ liệu ảnh khuôn mặt.

5 Kết luận và hướng phát triển

5.1 Kết luận

Trong đề án này, nhóm đã nghiên cứu và triển khai hai phương pháp giảm chiều dữ liệu: PCA (Phân tích Thành phần Chính) và Kernel PCA (KPCA). Các kết quả thực nghiệm trên hai bộ dữ liệu Olivetti và LFW đã chứng minh rằng:

- PCA là một phương pháp hiệu quả trong việc giảm chiều dữ liệu tuyến tính, đặc biệt khi dữ liệu có cấu trúc tuyến tính rõ ràng.
- Kernel PCA, với khả năng xử lý các mối quan hệ phi tuyến thông qua kernel trick, đã cho thấy ưu thế vượt trội trong việc trích xuất các đặc trưng quan trọng từ dữ liệu phi tuyến, giúp cải thiện đáng kể độ chính xác của bài toán nhận diện khuôn mặt.
- Việc lựa chọn số chiều tối ưu (d) và tham số kernel (như γ trong kernel RBF) đóng vai trò quan trọng trong hiệu quả của KPCA.

Kết quả thực nghiệm cho thấy KPCA đạt độ chính xác cao hơn PCA trên cả hai bộ dữ liệu, đặc biệt là trên bộ dữ liệu LFW phức tạp hơn. Điều này khẳng định tiềm năng của các phương pháp phi tuyến trong việc xử lý dữ liệu có cấu trúc phức tạp.

5.2 Hướng phát triển

Dựa trên các kết quả đạt được, chúng tôi đề xuất một số hướng phát triển trong tương lai:

- **Tối ưu hóa tham số kernel:** Thử nghiệm với các giá trị γ khác nhau hoặc sử dụng các phương pháp tự động tối ưu hóa tham số kernel như Grid Search hoặc Bayesian Optimization để cải thiện hiệu quả của KPCA.
- **Nghiên cứu các kernel khác:** Ngoài kernel RBF, có thể thử nghiệm các kernel khác như Polynomial, Sigmoid hoặc các kernel tùy chỉnh để phù hợp hơn với đặc điểm của dữ liệu.

- **Kết hợp với các phương pháp học sâu:** Sử dụng KPCA như một bước tiền xử lý để giảm chiều dữ liệu trước khi đưa vào các mô hình học sâu, nhằm giảm độ phức tạp tính toán và cải thiện hiệu suất.
- **Ứng dụng vào các bài toán khác:** Mở rộng ứng dụng của KPCA sang các lĩnh vực khác như phân tích dữ liệu y sinh, xử lý tín hiệu, hoặc thị giác máy tính để đánh giá tính tổng quát của phương pháp.
- **Xử lý dữ liệu lớn:** Nghiên cứu các biến thể của KPCA như Approximate Kernel PCA hoặc Randomized Kernel PCA để xử lý hiệu quả hơn các tập dữ liệu lớn.

Những hướng phát triển này không chỉ giúp cải thiện hiệu quả của KPCA mà còn mở rộng phạm vi ứng dụng của phương pháp trong các bài toán thực tế.

References

- [1] D. Precup, *Dimensionality Reduction. PCA. Kernel PCA.*, COMP-652 and ECSE-608, McGill University, March 14, 2016. Available: cs.mcgill.ca